DocQA 技术设计说明书

> 版本：v1.0 · 对应代码：`src/docqa/` · 更新日期：2026-10-24
>
> 本文回答三个问题：**系统怎么分层、层与层之间靠什么契约通信、关键算法如何实现。**

1. 设计范围与原则

1.1 范围

覆盖 DocQA 的五条主链路：

1. 多模态文档解析（`doc2md`）；
2. 语义切片与三路索引（`docbase.indexer`）；
3. 三路召回、RRF 融合与精排（`docbase.retriever`）；
4. Agent 问答编排与强制溯源（`agent.answer`）；
5. 可选的服务化入口（`service.watcher`、`web.server`）。

1.2 设计原则

原则	落地方式
分层单向依赖	`web/service → agent → docbase → doc2md → llm`，禁止反向 import
契约稳定	跨层只经 `models.py`；字段只增不改
模型可替换	四类端点统一由 `ModelHub` 注入；Mock 可离线跑通
证据优先	回答只使用检索到的 Chunk；不足即拒答
可核对	每个 Chunk 带 doc/page/heading/source；图片额外带 fig/title
可回滚	索引版本化；`index.json + vectors.json + sparse.json` 三件套
无重依赖	运行时仅标准库；PDF 解析可选 `pypdf`

2. 总体架构

```
┌──────────────────────────────────────────────────────────┐
│ 入口层  CLI / Web / Service(watcher)                     │
├──────────────────────────────────────────────────────────┤
│ Agent 层  answer_question: 意图 → 检索 → 自评 → 生成 → 引用  │
├──────────────────────────────────────────────────────────┤
│ 检索层  DocBase: chunk / build / query / RRF / rerank / eval │
├──────────────────────────────────────────────────────────┤
│ 解析层  doc2md: PDF/Word/PPT/MD → Markdown + 图片释义 + manifest│
├──────────────────────────────────────────────────────────┤
│ 模型端点层  VL / Embedding / Reranker / Chat (可 HTTP / Mock)  │
└──────────────────────────────────────────────────────────┘
```

依赖方向自上而下；`models.py` 是各层之间唯一的数据通道。

---

3. doc2md 解析层

3.1 入口与分发

```
parse_document(src, out_dir=None, category=None, hub=None, options=None) -> ParseResult
```

- 通过扩展名 + 文件头识别类型（PDF `%PDF`、OOXML `PK` + `word/` / `ppt/`）；
- 支持 `.md/.markdown`、`.pdf`、`.docx`、`.pptx`、`.txt/.text/.log`；
- 输出目录固定包含：
- `document.md`：带 front matter 的结构化 Markdown；
- `assets/`：按溯源规范重命名的图片；
- `manifest.json`：解析任务、图片清单、统计、警告。

3.2 Markdown 解析

逐行扫描：

1. `<!-- page: N -->` 更新当前页码；
2. `!alt` 解析本地图片：

- 相对路径 → 源文件目录；

- 复制到 `assets/`，按 `文档名_Page{N}_Fig{k}_{标题}.ext` 命名；
- 调 VL 释义；
- 在原位置替换为「图片 + 引用块 + 元信息注释」；

1. 标题、正文、表格原样保留；
2. front matter 重新生成（doc_id / source / pages / page_mode / category / parsed_at）。

图片写回格式：

```
![图题](assets/文档名_Page2_Fig1_图题.png)

> **图 2-1 · 图题**（VL 释义）：……图中模块与箭头……该图位于《文档名》第 2 页……

<!-- docqa:image path=assets/... page=2 figure=1 title=图题 -->
```

3.3 PDF 解析

- 使用 `pypdf` 逐页 `extract_text()`；PDF 取图需 `Pillow`；
- 每页写入 `<!-- page: N -->`，页码 exact；
- `page.images` 提取嵌入位图，写入当前页；
- 图题从页内最后几行中匹配 `图 N` / `Figure N`；
- 矢量图（PDF drawings）当前版本不直接渲染，记录告警；扩展点见 §11；
- 无 `pypdf` 时给出明确安装提示，不静默失败。

3.4 DOCX 解析

不依赖 `python-docx`，直接解析 OOXML：

- 读取 `word/document.xml`、`word/_rels/document.xml.rels`、`word/media/*`；
- 段落：`w:p` → `w:t` 拼接；`w:pStyle` 识别 Heading/标题；
- 图片：`a:blip` 的 `r:embed` → rels → media；
- 表格：`w:tbl` → GFM 表格；
- 分页：`w:br type="page"` 与 `w:lastRenderedPageBreak` 估算页码；
- `page_mode = estimated`，在 manifest/回答中明示。

3.5 PPTX 解析

- `ppt/slides/slideN.xml` 按编号排序，一页幻灯片 = 一页；
- 段落 `a:p` → `a:t`；第一段作为标题；

- 通过 `ppt/slides/_rels/slideN.xml.rels` 提取图片；
- 页码 exact，输出 `## Slide N` + 列表 + 图片块。

3.6 图片与溯源命名

```
文档名_Page{页码}_Fig{图序}_{Figure标题}.{ext}
```

- 图序按页内计数；
- 标题优先取图注（`图 1 ...`），否则取 alt；
- 移除路径分隔符、空格，保留中文；
- `ImageInfo.source_uri` 形如 `file:///.../doc.md#page=2&figure=1`；
- 图片释义与原文页码、图题、原图路径绑定，支持「图文互证」。

3.7 VL 释义协议

真实端点（OpenAI-compatible）：

```
POST {base}/chat/completions
{
  "model": "Qwen3.6-27B-VL",
  "messages": [{
    "role": "user",
    "content": [
      {"type": "text", "text": "请描述图中模块/箭头/编号/关键文字…"},
      {"type": "image_url", "image_url": {"url": "..."}}
    ]
  }]
}
```

Prompt 要求：客观描述模块、连线方向、编号步骤、关键数值；不臆造；220 字内。

Mock 模式：用作者提供的图注/alt + 版式关键词生成可复现释义，并在文末标注「离线 Mock 释义」。Mock 不替代真实读图能力，只保证离线链路与测试可跑。

3.8 失败与降级

场景	行为
图片不存在	保留原引用 + warning
EMF/WMF/SVG/TIFF	跳过读图 + warning（需先渲染）
VL 端点失败	回退 Mock 释义 + 记录 warning
PDF 取文/取图失败	该页为空 + warning，不中断整篇
不支持的格式	抛 `ValueError`，CLI 返回非 0

---

4. DocBase 检索层

4.1 语义切片

`chunk_markdown(text, doc_meta, images, chunk_size=1200, overlap=120)`

- 以标题与页码为边界：**一个 Chunk 不跨页**，保证引用页码准确；
- 按空行切段落；段落超长时按句子切；
- 代码块/Mermaid 保持原子性；若与前文合计不超限，则合并到同一 Chunk（让流程步骤与流程图同块）；
- Chunk 文本前缀 `【标题路径】`，保留章节语义；
- 附带该块引用到的图片元数据（`images`）；
- 过短块（< 20 字）丢弃，避免噪声。

Chunk 字段：

字段	类型	说明
`chunk_id`	str	`sha1(doc_id:order:text)[:16]`
`doc_id`	str	文档稳定标识
`doc_name`	str	文档名
`heading`	str	标题路径，如 `A / B / C`
`text`	str	含标题前缀与图片释义的检索文本
`page`	int	起始页码
`category`	str	分类
`source_path`	str	原始文档绝对路径
`source_uri`	str	`file://...#page=N`
`order`	int	文档内顺序
`images`	list	关联图片元数据
`content_hash`	str	文本哈希
`char_count`	int	字符数

4.2 Tokenizer

无第三方分词依赖，混合中英文：

- ASCII：`[a-zA-Z0-9][a-zA-Z0-9_.\-/#]*`，小写；
- 中文：连续汉字段的 **字符 bigram**（单字段保留单字）；
- 过滤停用词与疑问词。

同一 tokenizer 用于 BM25、稀疏向量、Mock Embedding 与 Mock Reranker，保证离线一致性。

4.3 稠密向量

- 真实模式：`POST {base}/embeddings`，模型 Qwen3-Embedding-8B；
- Mock 模式：512 维 hashing embedding：
- 对每个 token 做 MD5，取前 4 字节映射到维度，第 5 字节决定符号；
- 权重 `1 + ln(tf)`，L2 归一化；
- 优点：确定性、零依赖；缺点：**不具备真正语义泛化**，仅用于离线闭环。

相似度：余弦。

4.4 BM25

参数 `k1=1.5, b=0.75`：

```
IDF(t) = ln(1 + (N - df(t) + 0.5) / (df(t) + 0.5))
score(q, d) = Σ_t IDF(t) * tf(t,d) * (k1 + 1)
              / (tf(t,d) + k1 * (1 - b + b * |d| / avgdl))
```

其中 `N` 为 Chunk 数，`df` 为文档频率，`|d|` 为 Chunk token 数，`avgdl` 为平均长度。

4.5 稀疏向量

模拟 BGE-M3 的学习型稀疏表示：

```
w(t) = (1 + ln tf(t)) * IDF(t)
向量 L2 归一化
score = cosine(query_weights, chunk_weights)
```

存储为 `[[token, weight], ...]`，查询时用同一 IDF 计算 query 权重。

4.6 RRF 融合

每一路取前 50，按排名融合：

```
RRF(d) = Σ_r 1 / (60 + rank_r(d))
```

`k=60`；同一 Chunk 在多路出现则累加。融合后取前 `max(24, top_k*4)` 进入精排。

4.7 精排

- 真实模式：`POST {base}/rerank`，模型 Qwen3-Reranker-8B；
- Mock 模式：`0.62 * token 覆盖率 + 0.28 * 精确术语命中率 + 0.10 * 首段命中`；
- 最终分：

```
final = 0.72 * normalize(rerank) + 0.28 * normalize(RRF)
```

若 rerank 全为常数，则退化为 RRF 排序。

4.8 索引落盘

```
data/index/
├── index.json    # RagIndex：版本、文档注册表、Chunk 清单、统计
├── vectors.json  # {"model": "...", "dim": 512, "vectors": {chunk_id: [...]}}
└── sparse.json   # {"N":..., "avgdl":..., "df":..., "vectors":..., "tf":...}
```

- 增量建库：按文档内容哈希判断是否变更；未变更文档复用旧 Chunk；
- 向量按 `chunk_id` 复用（内容寻址）；Embedding 模型变化时强制重算；
- 删除文档时，其 Chunk 不再进入新索引；解析目录中的陈旧产物由 watcher 清理；
- 索引版本：`YYYY.MM.DD-HHMMSS`，可用于回滚与审计。

4.9 评估（test 模式）

`evaluate(index_dir, test_set, top_k=6)`：

- 每个用例：`question` / `expected_docs` / `expected_keywords` / `category?`；
- `recall@k = 命中期望文档数 / 期望文档数`；
- `hit_rate = 是否至少命中一个期望文档`；
- `MRR = 1 / 首个相关结果排名`；
- `keyword_hit_rate = 命中关键词数 / 关键词数`；
- 输出 per-case + summary，落盘 JSON。

---

5. Agent 层

5.1 问答主流程

```
answer_question(question, index_dir, max_rounds=2, top_k=6, category=None) -> Answer
```

1. **意图分类**：command / flow / version / troubleshoot / term / concept；
2. **Query 构造**：原问题 + 去疑问词/标点改写 + ASCII 术语补充；
3. **检索**：每个 query 调 DocBase，多 query 结果按 chunk_id 合并（取高分、RRF 累加）；
4. **充足性自评**：

- 最高 `rerank_score >= 0.20`；
- 问题 token 在 Top-3 Chunk 中的覆盖率 `>= 0.30`；

- 不满足则用 Top-1 的标题扩展 query，进入第二轮；

1. **生成**：

- 真实模式：GLM-5.1，严格 system prompt；
- Mock 模式：抽取式合成（结论 + 步骤/表格/句子 + 引用）；

1. **结构化**：流程类附加 Mermaid；命令/参数类附表格；版本类附特性表；
2. **引用重编号**：只保留回答正文实际使用的引用，按首次出现重排 `[n]`；
3. **返回** `Answer`。

5.2 意图与检索策略

意图	触发词示例	优先证据形态
command	命令、参数、配置、操作、CLI	命令表、注意事项
flow	流程、步骤、转发、接续、时序	编号步骤、Mermaid
version	版本、特性、发布、变更	特性表
troubleshoot	故障、排查、失败、告警	排查步骤
term	含型号/英文术语	术语定义、命令
concept	其他	段落/列表

5.3 抽取式合成规则

Mock 模式不调用 Chat，仍要保证可读与可核对：

- **流程**：取原文编号步骤 3 条以上 → 作为「关键依据」；原文有 Mermaid 则重绘；
- **排障**：取编号步骤，结论为「建议按以下步骤排查」；
- **命令**：选含「命令/反引号/reset」的句子作结论，附执行前注意事项；
- **参数**：优先表格行，结论取参数定义句；
- **版本**：取「新增特性」表行，按 `特性：说明（受益场景）` 格式化；
- **概念**：按 query token 覆盖率选句，去重后输出。

5.4 引用策略

- 每个 `[n]` 对应一个真实 `Chunk`；
- `Citation` 字段：`ref / doc_id / doc_name / page / heading / snippet / source_uri / source_path / figure`；
- 图片优先使用 `ImageInfo.source_uri`（带 `#page=&figure=`）；

- 回答正文只保留实际出现的引用，`溯源` 段按新编号重建；
- 无依据时 `citations=[]`，明确输出「无据不答」。

5.5 Prompt 约束

```
你是 DocQA……
1. 存在性：本仓库配备 DocBase 库，回答前必须先检索；
2. 查询策略：技术名词优先精确检索，原理/流程优先语义检索；
3. 溯源强制：每个结论标注 [n]，给出文档名/页码/图号；不足时回答「无据不答」，禁止编造；
4. 产出形式：流程类给出 Mermaid，并提示与原图对照。
只依据提供资料回答。
```

---

6. 服务化入口

6.1 CLI

命令	作用
`parse`	解析单文件/目录
`build`	建库（`--incremental`）
`query`	只检索，显示三路分数
`ask`	检索 + 生成 + 溯源；`--show-trace`
`eval`	recall@k / hit_rate / MRR
`serve`	Web 问答
`watch`	目录监控 + 增量更新
`demo`	一键跑通
`doctor`	环境/模型/索引检查

6.2 Web API

方法	路径	说明
GET	`/api/health`	存活检查
GET	`/api/stats`	索引统计、文档列表、模型模式
GET	`/api/doc?doc_id=&mode=parsed`	查看解析后原文
GET	`/api/search?q=&top_k=`	只检索
POST	`/api/ask`	`{question, top_k?, max_rounds?, category?}` → `WebQueryResponse`

`WebQueryResponse` 与 `answer_question` 对齐：`answer / format / citations / rounds_used / sufficient / elapsed_ms / trace / mermaid`。

6.3 Watcher

- 轮询目录（默认 10 s），比较 `(mtime, size)` 快照；
- 变更文件调用 `parse_document`，删除文件清理对应解析目录；
- 每轮调用 `build_index(incremental=True)`；
- 事件写 `data/watch.log`（JSONL）；
- `--once` 便于测试与手工触发。

---

7. 数据契约汇总

7.1 doc2md

契约	关键字段
`ImageInfo`	image_id, doc_name, page, figure_index, title, path, kind, width, height, description, caption, source_ref, source_uri, vl_model, vl_mode
`ParseTask`	task_id, src, out_dir, doc_type, category, vl_mode, options, created_at
`ParseResult`	task, doc_id, doc_name, markdown_path, manifest_path, assets_dir, images, warnings, stats, pages, page_mode, source_uri

7.2 docbase

契约	关键字段
`Chunk`	chunk_id, doc_id, doc_name, heading, text, page, category, source_path, source_uri, order, images, content_hash, char_count
`RagIndex`	version, created_at, updated_at, embedding_model, rerank_model, chunk_size, overlap, docs, chunks, stats, schema_version
`QueryHit`	chunk, score, rerank_score, dense_score, bm25_score, sparse_score, rrf_score, rank, routes
`QueryResult`	query, hits, latency_ms, route_stats, warnings, index_version, category

7.3 agent / web

契约	关键字段
`Citation`	ref, doc_id, doc_name, page, heading, snippet, source_uri, source_path, figure
`Answer`	question, answer, format, citations, rounds_used, sufficient, queries, trace, mermaid, elapsed_ms, index_version, model_mode, warnings
`WebQueryResponse`	question, answer, format, citations, rounds_used, sufficient, elapsed_ms, index_version, model_mode, trace, mermaid, warnings

兼容规则：新增字段必须带默认值；不得重命名/删除既有字段；如需破坏性变更，提升 `schema_version` 并提供迁移脚本。

---

8. 模型端点协议

8.1 配置

```
DOCQA_MODEL_MODE=http
DOCQA_BASE_URL=https://model-gateway.example.com/v1
DOCQA_API_KEY=***
DOCQA_CHAT_MODEL=GLM-5.1
DOCQA_VL_MODEL=Qwen3.6-27B-VL
DOCQA_EMBED_MODEL=Qwen3-Embedding-8B
DOCQA_RERANK_MODEL=Qwen3-Reranker-8B
```

8.2 端点约定

能力	方法	路径	响应关键字段
Chat	POST	`/chat/completions`	`choices[0].message.content`
VL	POST	`/chat/completions`	同上（content 含 image_url）
Embedding	POST	`/embeddings`	`data[].embedding`
Rerank	POST	`/rerank`	`results[].{index, relevance_score}`

- 任一调用失败：记录 warning，后续请求自动切换 Mock（`_http_failed`），避免反复超时；
- 端点不含 `/v1` 时由 `BASE_URL` 自行配置；代码只拼接相对路径；
- 不在代码/日志中输出 API Key。

8.3 Mock 边界（必须明示）

Mock 仅保证：

- 端到端链路可跑通；
- 单元/集成测试可复现；
- 输出格式与真实模式一致。

Mock **不保证**：

- 语义泛化能力；
- 真实图片理解能力；
- 生产检索质量指标。

因此项目书中的 1.0 指标仅对演示集与 Mock 链路负责；生产上线必须切换真实端点并重建索引。

---

9. 安全与隐私

面	措施
数据	文档、索引、图片全部落在本机 `data/`；不上传公网
模型	仅调用配置的内部端点；默认 Mock 不发起网络请求
Web	默认绑定 `127.0.0.1`；文档读取只按 `doc_id` 查注册表，不接受任意路径
密钥	只读环境变量；不写入索引/日志/前端
依赖	运行时标准库；PDF 仅可选 pypdf；减少供应链面
审计	`trace` 记录 query、命中数、覆盖率、耗时；`watch.log` 记录变更

10. 性能与容量

环节	当前实现	复杂度/备注
解析	逐文件、逐页	线性；VL 调用是主要时延
切片	线性扫描	O(字符数)
建库	批量 embedding（16/批）	线性；增量复用
稠密召回	全量余弦	O(N · d)；万级 Chunk 可换向量库
BM25	倒排 tf/df	O(查询词 × 候选)
稀疏	稀疏点积	O(非零项)
精排	候选 24 – 40 条	真实 Reranker 时延为主
存储	JSON	演示 32 块约 0.6 MB；规模大时换 SQLite/向量库

演示数据：5 文档 / 32 Chunk / 3 图片；Mock 检索约 2 ms，问答约 9 – 15 ms。

11. 扩展点

1. **PDF 矢量图渲染**：接入 PyMuPDF，对含 drawings 的页面渲染为位图后送 VL；保持 `ImageInfo` 契约不变。
2. **向量库替换**：`Retriever` 的稠密召回替换为 FAISS/Milvus/pgvector，接口仍返回 `{chunk_id: score}`。

3. **真实 BGE-M3 稀疏**：将 `sparse.json` 的模拟权重替换为模型输出的 `{token: weight}`，RRF/精排不变。
4. **无头 Agent 服务**：`web.server` 的 `/api/ask` 内部改调 `claude -p` / `nga run`，`WebQueryResponse` 不变。
5. **多用户/权限**：在入口层增加租户与 ACL；索引按租户分区；下层契约不变。
6. **OCR 兜底**：扫描版 PDF 先走 OCR（PaddleOCR/Tesseract），再走现有解析链路。

---

12. 已知限制

- `.doc`（OLE 旧格式）未直接支持，需先转 `.docx`；
- DOCX 无固定分页，页码为估算值，已在 `page_mode` 与回答中标注；
- 矢量图（EMF/WMF/SVG/TIFF）当前跳过读图并告警；
- Mock Embedding 基于词面哈希，不等价于语义模型；
- 演示语料为虚构的 FE_HEART 场景，仅用于验证系统能力，不代表真实产品文档；
- Web 为单机本地服务，未做多用户鉴权与并发压测。

---

13. 测试策略

层级	用例	断言
doc2md	Markdown 图文解析、重复解析、TXT、最小 DOCX、最小 PPTX	页码/图序/命名/释义/manifest
docbase	建库、三路召回、精确术语、分类过滤、评估、增量更新	命中文档、页码、指标、更新计数
agent	流程问答、命令问答、无据拒答、契约字段、意图分类	引用、Mermaid、`sufficient`、字段
web	静态页、stats、ask、doc	HTTP 200、JSON 契约、原文内容

执行：

```
make test
# 或
PYTHONPATH=src:tests python3 -m unittest discover -s tests -v
```

当前 19 项全部通过。